Optimization Algorithms: Heuristics, Metaheuristics, and Approximations

Approximations

Strategies for NP-Hard Combinatorial Problems

Context: Exact algorithms guarantee optimality but scale exponentially for NP-hard problems. This presentation explores non-exact methodologies—Approximation Algorithms, Heuristics, and Metaheuristics—that trade absolute optimality for computational feasibility, scalability, and robust practical performance.

Transition Motivation: *To navigate these methodologies, we first establish the structural roadmap of this presentation.*

Presentation Outline

1. Approximation Algorithms

- Theoretical Framework and Bounds
- Christofides' Algorithm

2. Heuristics

- Constructive vs. Improvement Strategies
- Numerical Example: Nearest Neighbor TSP

3. Metaheuristics

- Exploration vs. Exploitation
- Trajectory-based Frameworks (SA, TS, VNS, ILS)
- Population-based Frameworks (GA, ACO, PSO)
- Hybrid Frameworks (GRASP)

4. Comparative Analysis

- Decision Matrix and Trade-offs

Transition Motivation: We begin with Approximation Algorithms, which provide the strongest theoretical guarantees among non-exact methods.

Section 1: Approximation Algorithms

1. Approximation Algorithms

- Theoretical Framework and Bounds
- Christofides' Algorithm

2. Heuristics

3. Metaheuristics

4. Comparative Analysis

Transition Motivation: Let us define what qualifies an algorithm as an "approximation" mathematically.

Theoretical Framework and Approximation Bounds

An approximation algorithm is a polynomial-time algorithm for an NP-hard optimization problem that guarantees a bounded distance from the true optimum (OPT).

Approximation Ratio (ρ)

- Minimization: Algorithm returns cost $C \leq \rho \cdot \mathrm{OPT}$ (where $\rho \geq 1$).
- Maximization: Algorithm returns profit $C \geq \rho \cdot \mathrm{OPT}$ (where $\rho \leq 1$).
- The ratio ρ measures the worst-case solution quality relative to the absolute optimum.

Polynomial-Time Schemes

- PTAS (Polynomial-Time Approximation Scheme):
  - Algorithm parameterized by error bound $\epsilon > 0$.
  - Returns solution within $(1 + \epsilon)$ of OPT.
  - Runs in polynomial time for any fixed ϵ (e.g., $\mathrm{O}(n^{1/\epsilon})$).
- FPTAS (Fully PTAS):
  - Stronger formulation; execution time is bounded by a polynomial in both the input size n and $1/\epsilon$.

Transition Motivation: To understand how these bounds are achieved in practice, we examine a classic approximation algorithm for the Traveling Salesperson Problem (TSP).

Christofides' Algorithm for Metric TSP

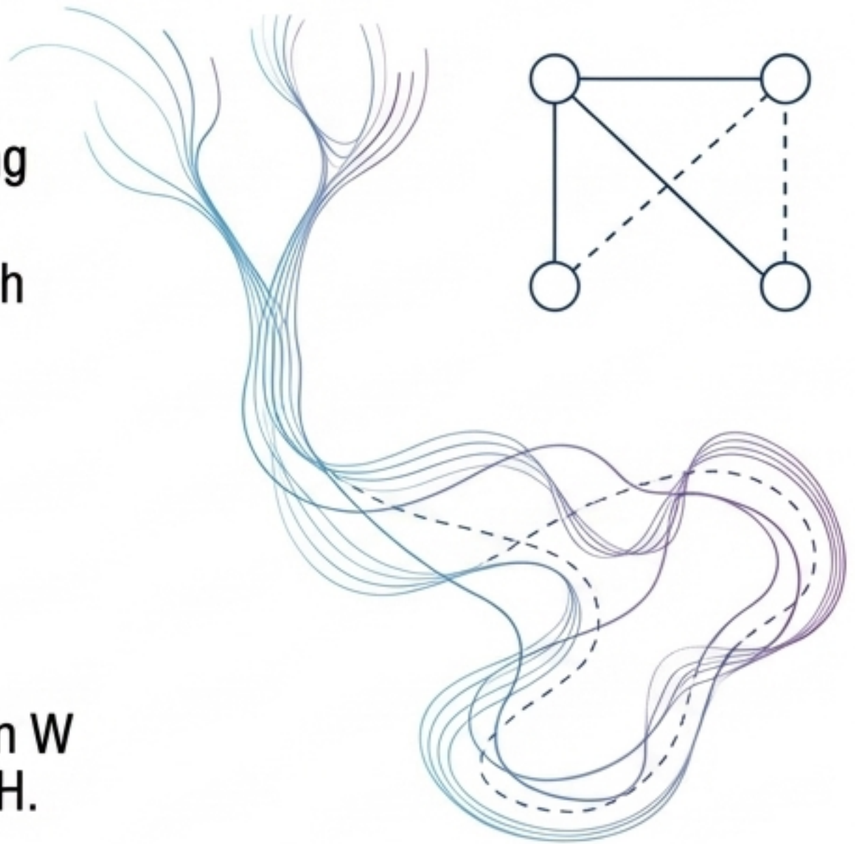
Applies to Metric TSP where the triangle inequality holds. It improves upon the standard 2-approximation Double-Tree approach.

Theoretical Basis & Bound

- **Mathematical Bound:** Guarantees a 3/2-approximation ($\rho = 1.5$).
- **Proof Logic:**
  - Weight of Minimum Spanning Tree (MST) $T \leq$ OPT.
  - Weight of perfect matching M on odd vertices $\leq$ 1/2 OPT.
  - Total Eulerian multigraph weight $w(T \cup M) \leq$ 3/2 OPT.

Algorithm Steps

1. Compute Minimum Spanning Tree (MST), T.
2. Identify set O of vertices with odd degrees in T.
3. Compute minimum-weight perfect matching M on O.
4. Combine to form Eulerian multigraph $T \cup M$.
5. Compute an Euler tour W.
6. Shortcut repeated vertices in W to return Hamiltonian cycle H.



Transition Motivation: While approximation algorithms offer strict bounds, their design is highly problem-specific. For broader applicability, we shift to general Heuristics.

Section 2: Heuristics

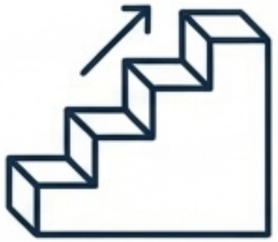
1. Approximation Algorithms
2. **Heuristics**
   - Constructive vs. Improvement Strategies
   - Numerical Example: Nearest Neighbor TSP
3. Metaheuristics
4. Comparative Analysis

Transition Motivation: We will define the two primary categories of heuristic algorithms and their operational differences.

Constructive vs. Improvement Strategies

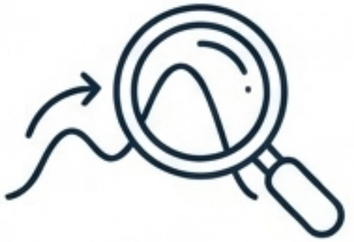
Heuristics are problem-solving methods designed to quickly produce a feasible solution, sacrificing guaranteed optimality for computational speed and scalability.

Constructive Heuristics



- **Mechanism**: Build a solution from an empty state by adding components sequentially based on a greedy rule.
- **Example**: Greedy Knapsack (sort by value-to-weight ratio), Nearest Neighbor TSP.
- **Limitation**: Highly sensitive to the initial construction order; early local choices restrict global optimality and cannot be undone.

Search-Based (Improvement) Heuristics



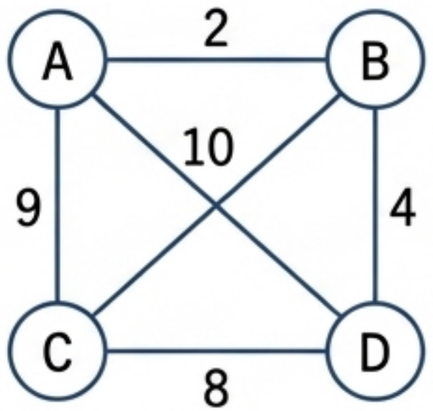
- **Mechanism**: Begin with a complete feasible solution and repeatedly apply local modifications.
- **Components**: Defines a neighborhood structure (e.g., 2-opt swap for routing).
- **Limitation**: Prone to stagnation at local optima (a solution strictly better than all its immediate neighbors, but sub-optimal globally).

Transition Motivation: To demonstrate the mechanics and limitations of a constructive heuristic, we will execute a step-by-step numerical example.

Numerical Example: Nearest Neighbor Heuristic

Problem Statement: Solve Metric TSP for 4 nodes (A, B, C, D).
Objective: Minimize total travel distance.
Components: Distance matrix D, Start Node = A, Unvisited Set U.

Node	A	B	C	D
A	0	2	9	10
B	2	0	6	4
C	9	6	0	8
D	10	4	8	0



Step-by-Step Execution

1. Start at A. U = {B, C, D}.
2. Nearest to A is B (d=2). Move to B. U = {C, D}.
3. Nearest to B in U is D (d=4). Move to D. U = {C}.
4. Nearest to D in U is C (d=8). Move to C. U = {}.
5. Return to start A (d=9).

Heuristic Tour: A → B → D → C → A. Total Distance = 23.

Differences (Comparison to Optimal):

- Optimal Tour: A → B → C → D → A (2 + 6 + 8 + 10 = 26).

(Note: In this specific matrix, the heuristic found a local optimum of 23. However, exact global evaluation proves greedy choices do not guarantee the absolute mathematical minimum for all matrix variations, as local choices can easily force a massive penalty on the final return leg).

Transition Motivation: Because basic heuristics get trapped in local optima, we require high-level frameworks capable of escaping them: Metaheuristics.

Source: Answers of the Presentation

Section 3: Metaheuristics

- 1. Approximation Algorithms
- 2. Heuristics
- **3. Metaheuristics**
  - Exploration vs. Exploitation
  - Frameworks (SA, GA, Tabu, ACO, PSO, GRASP, VNS, ILS)
- 4. Comparative Analysis

Transition Motivation: Before exploring specific algorithms, we must define the core trade-off that governs all metaheuristics.

The Core Paradigm: Exploration vs. Exploitation

Metaheuristics are high-level algorithmic frameworks combining generic and problem-specific rules to effectively explore large search spaces and escape poor local optima.

Exploitation (Intensification)

- **Definition**: Searching thoroughly within a promising, localized region of the solution space.
- **Goal**: Refines current good solutions to find the exact peak of a local basin (e.g., Local Search).
- **Risk**: Premature convergence on a sub-optimal peak.

Exploration (Diversification)

- **Definition**: Searching entirely new, unvisited regions of the global solution space.
- **Goal**: Prevents stagnation and discovers new basins of attraction (e.g., Random Restarts, Perturbations).
- **Risk**: Wasting computational cycles on barren regions.

Metaheuristic Architecture: Successful metaheuristics actively balance these two forces using memory, probabilistic acceptance rules, and dynamic neighborhood structures.

Transition Motivation: We will now analyze 8 distinct metaheuristic algorithms. Each will be visually deconstructed into its Mathematics, Pseudocode, and Flowchart.

Simulated Annealing (SA)

Mimics physical metallurgy. Uses probabilistic acceptance to escape local minima.

Mathematics

- Objective: Minimize $f(s)$
- Delta: $\Delta = f(s') - f(s)$
- Acceptance Probability:

$$P = 1 \quad \text{if } \Delta \leq 0$$

$$P = e^{-\Delta/T} \quad \text{if } \Delta > 0$$

- Parameters:

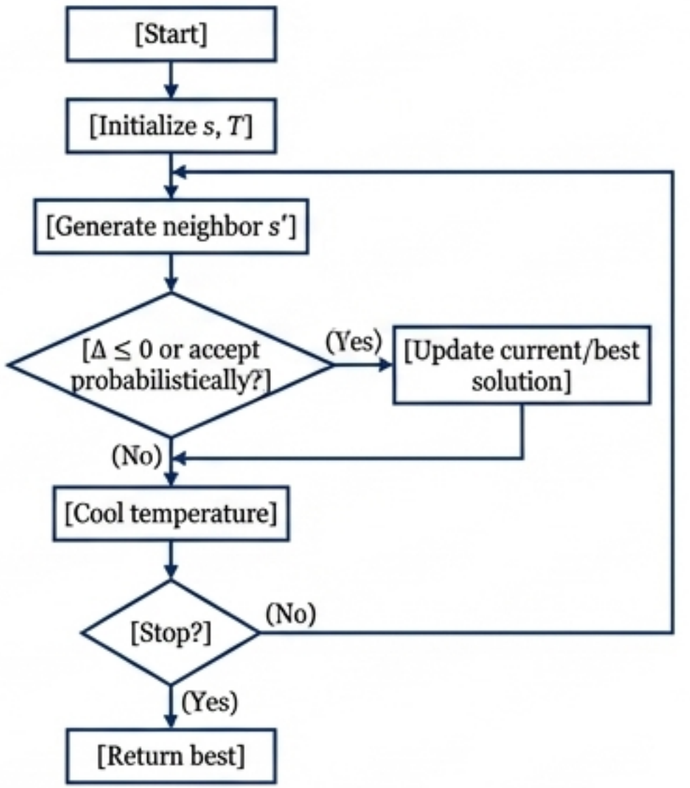
  T_0 (Initial Temp)

  α (Cooling rate, $T \leftarrow \alpha T$)

Pseudocode

```
1: Init s, T
2: s* ← s
3: While not Stop:
4:   s' ← Neighbor(s)
5:   Δ ← f(s') - f(s)
6:   If Δ <= 0 or rand() < exp(-Δ/T):
7:     s ← s'
8:     If f(s) < f(s*): s* ← s
9:   T ← Cool(T)
10: Return s*
```

Flowchart



Transition Motivation: While SA evolves a single solution, Population-based methods like Genetic Algorithms evolve multiple solutions concurrently.

Genetic Algorithms (GA)

Maintains a population of solutions, evolving them via biologically inspired operators.

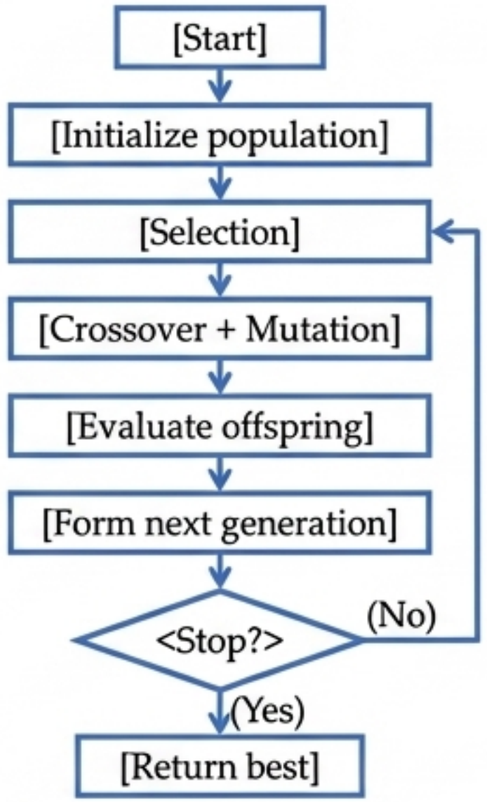
Mathematics

- **Representation:** Chromosome encoding (e.g., binary $x \in \{0,1\}^n$)
- **Fitness Evaluation:** $F(x) = f(x)$ or transformed variant.
- **Parameters:**
  N (Population Size)
  P_c (Crossover Rate)
  P_m (Mutation Rate)

Pseudocode

```
1: Init Population P
2: Evaluate P
3: While not Stop:
4:    Select parents from P
5:    Apply Crossover for
   offspring
6:    Apply Mutation
7:    Evaluate offspring
8: P ← Next Generation
9: Return Best in P
```

Flowchart



Transition Motivation: GA relies on randomness, whereas Tabu Search relies on strict memory structures to force exploration.

Tabu Search (TS)

Local search utilizing adaptive memory (Tabu list) to prevent cycling back to recent solutions.

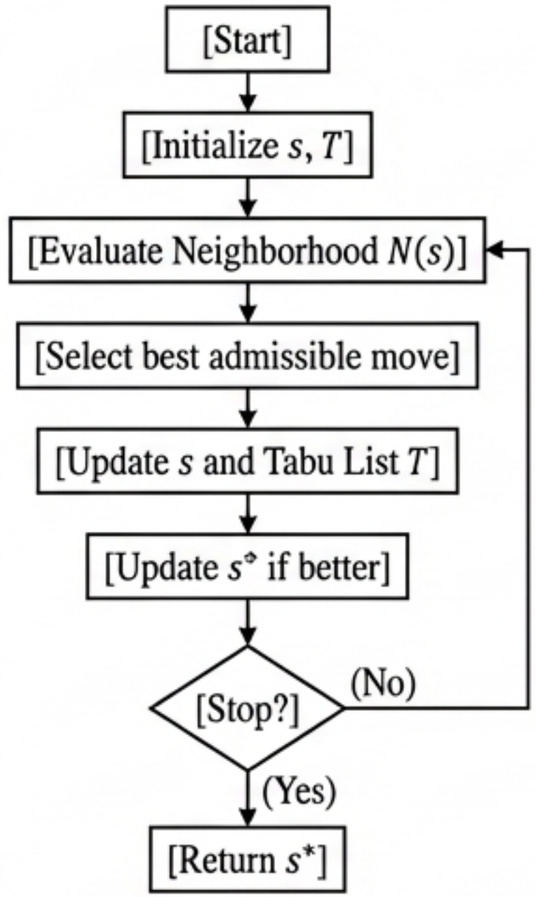
Mathematics

- Neighborhood: $N(s)$
- Tabu List (T): Set of forbidden moves.
- Aspiration Criterion: $f(s') < f(s^*)$ overrides Tabu status.
- Parameters: $|T|$ (Tabu Tenure - memory lifespan)

Pseudocode

1: Init s, $s^* \leftarrow s$, $T \leftarrow \emptyset$
2: While not Stop:
3: Gen Neighborhood $N(s)$
4: **Find best** s' **in** $N(s)$
where $s' \notin T$ or Aspiration(s')
5: $s \leftarrow s'$
6: **Update Tabu List** T
7: **If** $f(s) < f(s^*)$**:** $s^* \leftarrow s$
8: Return s^*

Flowchart



Transition Motivation: We move from individual memory to collective stigmergic memory with Ant Colony Optimization.

Ant Colony Optimization (ACO)

Probabilistic construction framework utilizing collective stigmergy (pheromone trails).

Mathematics

- Transition Probability:

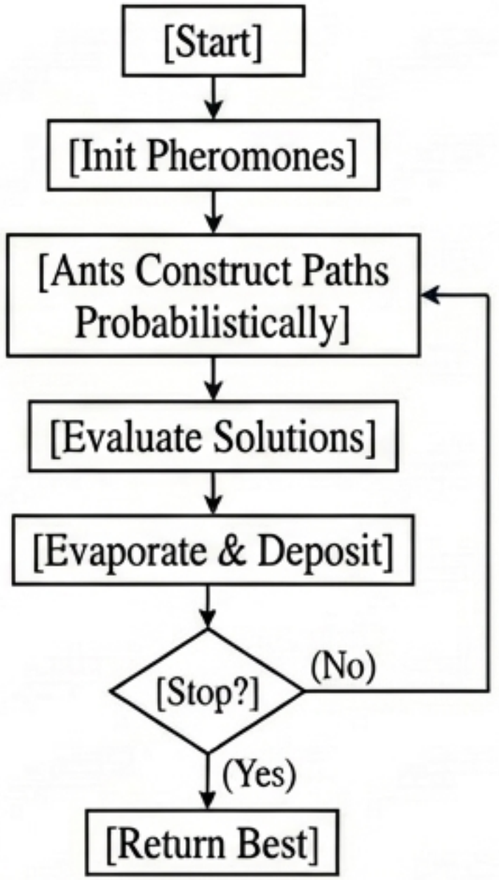
$$P_{ij} = \frac{(\tau_{ij})^{\alpha}(\eta_{ij})^{\beta}}{\sum(\tau_{ik})^{\alpha}(\eta_{ik})^{\beta}}$$

- Pheromone Update:
  $\tau_{ij} \leftarrow (1-\rho)\tau_{ij} + \Delta\tau_{ij}$
- Parameters:
  α (Pheromone weight)
  β (Heuristic weight)
  ρ (Evaporation rate)

Pseudocode

1: Init pheromone trails τ
2: While not Stop:
3: **For** each ant k:
4: Construct solution (P_{ij})
5: Evaluate solution
6: Evaporate pheromones (ρ)
7: Deposit pheromones ($\Delta\tau$)
8: Return Best Solution

Flowchart



Transition Motivation: Another collective swarm intelligence paradigm operates in continuous vector spaces: Particle Swarm Optimization.

Particle Swarm Optimization (PSO)

Population moves in search space guided by individual and collective best positions.

Mathematics

- Velocity Update:

$$v_i \leftarrow w \cdot v_i + c_1 \cdot r_1 (p_i - x_i) + c_2 \cdot r_2 (g - x_i)$$

- Position Update:

$$x_i \leftarrow x_i + v_i$$

- Parameters:

  w (Inertia weight)

  c_1 (Inertia weight)

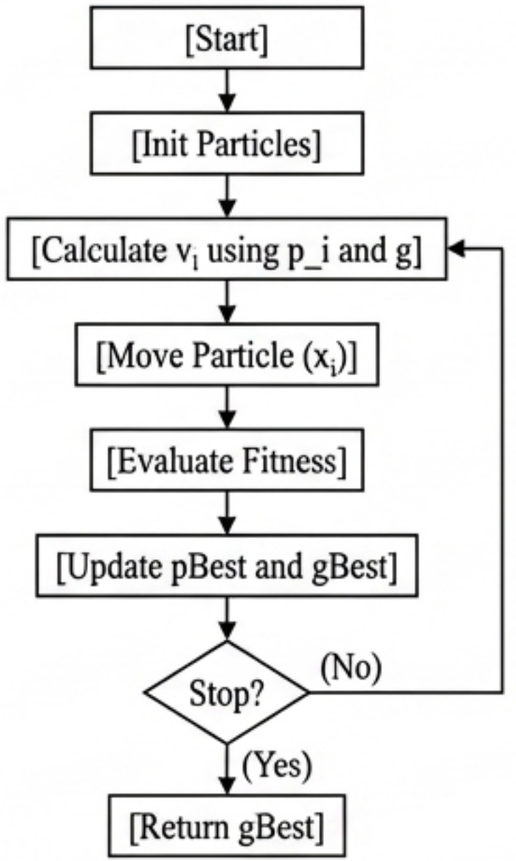
  c_1 (Cognitive coefficient)

  c_2 (Social coefficient)

Pseudocode

```
1: Init Positions X, Velocities V
2: Eval X, record pBest, gBest
3: While not Stop:
4:    For each particle i:
5:       Update velocity v_i
6:       Update position x_i
7:       Evaluate x_i
8:       Update pBest_i, gBest
9: Return gBest
```

Flowchart



Transition Motivation: Moving away from pure swarms, hybrid approaches combine constructive greediness with local improvement.

Source: Answers of the Presentation_5th.pdf, p. 13

GRASP (Greedy Randomized Adaptive Search Procedure)

Alternates randomized multi-start construction with local search intensification.

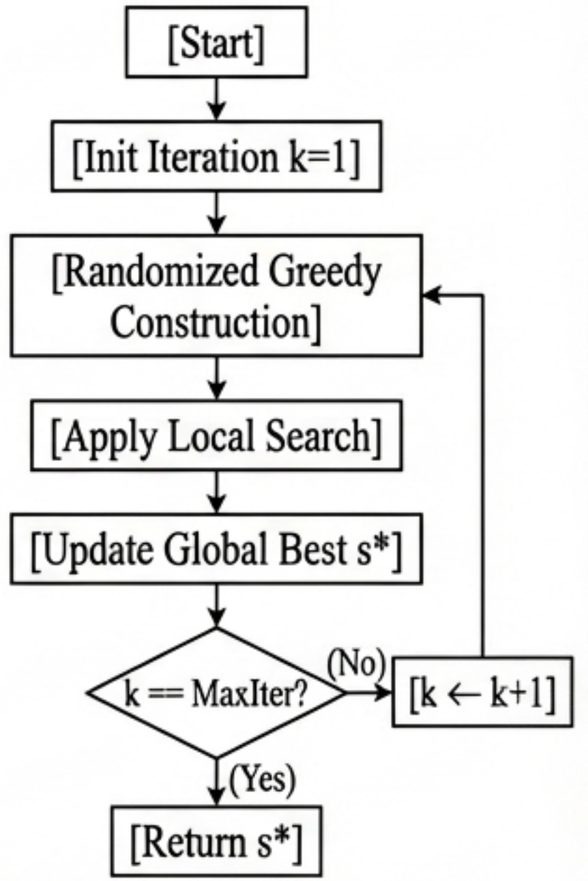
Mathematics

- Restricted Candidate List (RCL): Set of best elements evaluated by greedy function $g(x)$.
- Randomization: Select element $e \in RCL$ uniformly.
- Parameters:
  α (Threshold limit for RCL)
  $MaxIter$ (Number of full construction passes)

Pseudocode

```
1: s* ← NULL
2: For k = 1 to MaxIter:
3:     s ← RandomizedGreedy()
           // Build using RCL
4:     s' ← LocalSearch(s)
5:     If f(s') < f(s*):
6:         s* ← s'
7: Return s*
```

Flowchart



Transition Motivation: While GRASP alters construction boundaries, Variable Neighborhood Search alters the boundaries of the local search itself.

Source: Answers of the Presentation

Variable Neighborhood Search (VNS)

Systematic exploitation of multiple, nested neighborhood structures to escape stagnation.

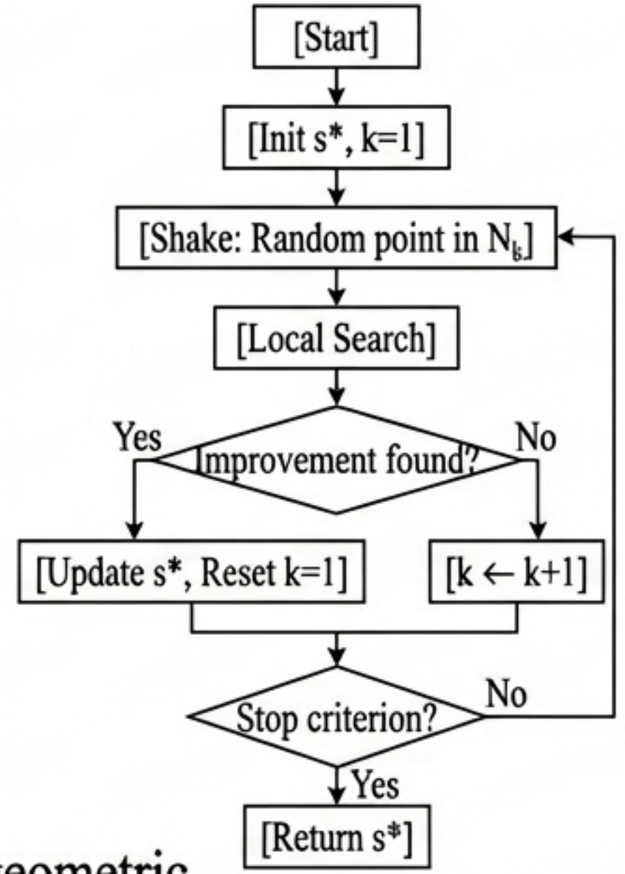
Mathematics

- Neighborhood Set: N_k, where $k = 1, 2, \ldots, k_{max}$.
- Shaking Phase: Select random $s' \in N_k(s^*)$.
- Local Search: Yields s'' from s'.
- Parameters: k_{max} (Max neighborhood size/strength).

Pseudocode

```
1: Init s, s* ← s
2: While not Stop:
3:     k ← 1
4:     While k ≤ k_max:
5:         s' ← Shake(N_k(s*))
6:         s'' ← LocalSearch(s')
7:         If f(s'') < f(s*):
8:             s* ← s'', k ← 1
9:         Else: k ← k + 1
10: Return s*
```

Flowchart



Transition Motivation: A closely related concept uses distinct geometric perturbations rather than multiple predefined neighborhoods: Iterated Local Search.

Source: Answers of the Presentation

Iterated Local Search (ILS)

Alternates between local search and controlled perturbation to jump between solution basins.

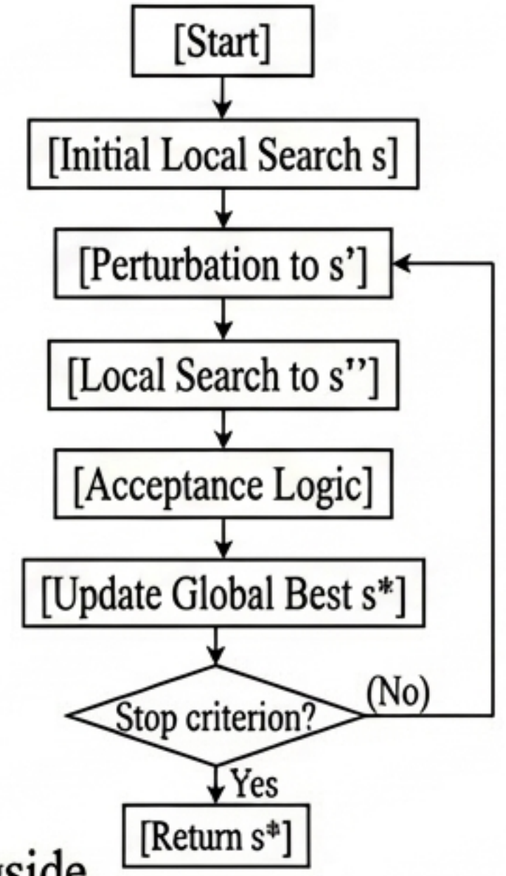
Mathematics

- Perturbation Strength: Must be mathematically large enough to escape the local optimum basin, but small enough to retain good historical attributes.
- Acceptance Criterion: Logic defining if the perturbed, optimized solution replaces the current state.
- Components: Perturb(), LocalSearch().

Pseudocode

```
1: s0 ← GenerateInit()
2: s ← LocalSearch(s0)
3: s* ← s
4: While not Stop:
5:     s' ← Perturb(s)
6:     s'' ← LocalSearch(s')
7:     s ← Accept(s, s'')
8:     If f(s) < f(s*): s* ← s
9: Return s*
```

Flowchart



Transition Motivation: Having detailed eight major metaheuristics alongside exact and approximation algorithms, we require a framework to compare and select them.

Source: Answers of the Presentation

Section 4: Comparative Analysis

1. Approximation Algorithms
2. Heuristics
3. Metaheuristics
4. **Comparative Analysis**
    - Decision Matrix and Trade-offs

Transition Motivation: ***The choice of algorithm depends heavily on operational constraints: available time, data scale, and the necessity of mathematical optimality.***

Decision Matrix: Algorithmic Trade-offs

Method	Solution Quality	Time Complexity	Scalability	Theoretical Guarantees
Exact Algorithms	Absolute Optimal	Exponential (for NP-hard)	Weak	Strongest (Provable Optimality)
Approximation	Near-optimal	Polynomial	Good	Strong (Provable Ratio Bound)
Heuristics	Good in practice	Very Fast	Very Good	Usually none
Heuristics	Good in practice	Very Fast	Very Good	Usually none
Metaheuristics	Very good / Near-optimal	Moderate to High (Iterative)	Very Good	None to Weak (Probabilistic)

Strategic Application Guidelines

- Use **Exact Methods** for small instances or mission-critical bounds.
- Use **Approximation** when strict bounds are legally or operationally required at scale.
- Use **Heuristics** for real-time decision-making systems (e.g., millisecond network routing).
- Use **Metaheuristics** for complex, large-scale problems where exactness fails but high solution quality is paramount.

Transition Motivation: *We conclude by distilling these algorithmic insights into foundational principles.*

Conclusion & Key Insights

1. **Combinatorial Intractability:** NP-hard problems require unavoidable compromises between computational budget and solution quality.

2. **Mathematical Precision:** Approximation algorithms offer the only polynomial-time mathematical guarantees for sub-optimal solutions.

3. **Local Optima Traps:** Basic constructive heuristics are fast but inherently blind to the global topography of the design space.

4. **Intelligent Navigation:** Metaheuristics utilize adaptive memory, randomness, and population dynamics to actively balance exploration and exploitation.

5. **No Free Lunch:** No single methodology dominates. The optimal algorithmic selection is always a strict function of instance size, time constraints, and the required rigor of the solution.

End of Presentation.